PRNG Lab

Name: Matthew Blanchard
Date: February 16, 2026
Professor: Dr. Joshuva
Course: CYEN 4335-01

Executive Summary

This lab explores weak versus secure pseudo-random number generation, predictability of time-seeded PRNGs, Linux entropy measurement, differences between /dev/random and /dev/urandom, and secure cryptographic key generation using the SEED Ubuntu 20.04 VM.

Task 1

Objective

Generate a 128-bit key with a PRNG using time as the seed.

Instructions

1. Compile and execute the provided C program that generates a 128-bit key using srand(time(NULL)).
2. Run the program multiple times within the same second. Then comment out the srand() line and run again.
3. Explain the role of time() and srand(), and why this method is insecure for cryptographic purposes.

Observations

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define KEYSIZE 16

void main()
{
   int i;
   char key[KEYSIZE];

   printf("%llld\n", (long)time(NULL));
   srand (time(NULL));

   for (i=0; i<KEYSIZE; i++){
      key[i] = rand()%256;
      printf("%.2x", (unsigned char)key[i]);
   }
   printf("\n");
}
~
~
~
~
"wrongprng.c" 20L, 315C                                    6,0-1         All
```

Figure 1.1: Source code for PRNG.

```
[02/13/26]seed@VM:~/.../cryptography$ vim wrongprng.c
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002282
5c35390cee3a9a7e2a9dfc894437dbfd
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002282
5c35390cee3a9a7e2a9dfc894437dbfd
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002283
ee9be7fd45a5a7e8439eceaed6674598
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002283
ee9be7fd45a5a7e8439eceaed6674598
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002284
dfeab9984e21a6d7bb3bd061f7b44c36
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002284
dfeab9984e21a6d7bb3bd061f7b44c36
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002284
dfeab9984e21a6d7bb3bd061f7b44c36
[02/13/26]seed@VM:~/.../cryptography$ ./wrongprng
 1771002284
dfeab9984e21a6d7bb3bd061f7b44c36
```

Figure 1.2: Output of PRNG program running in rapid succession.

Figure 1.3: Commenting out srand().

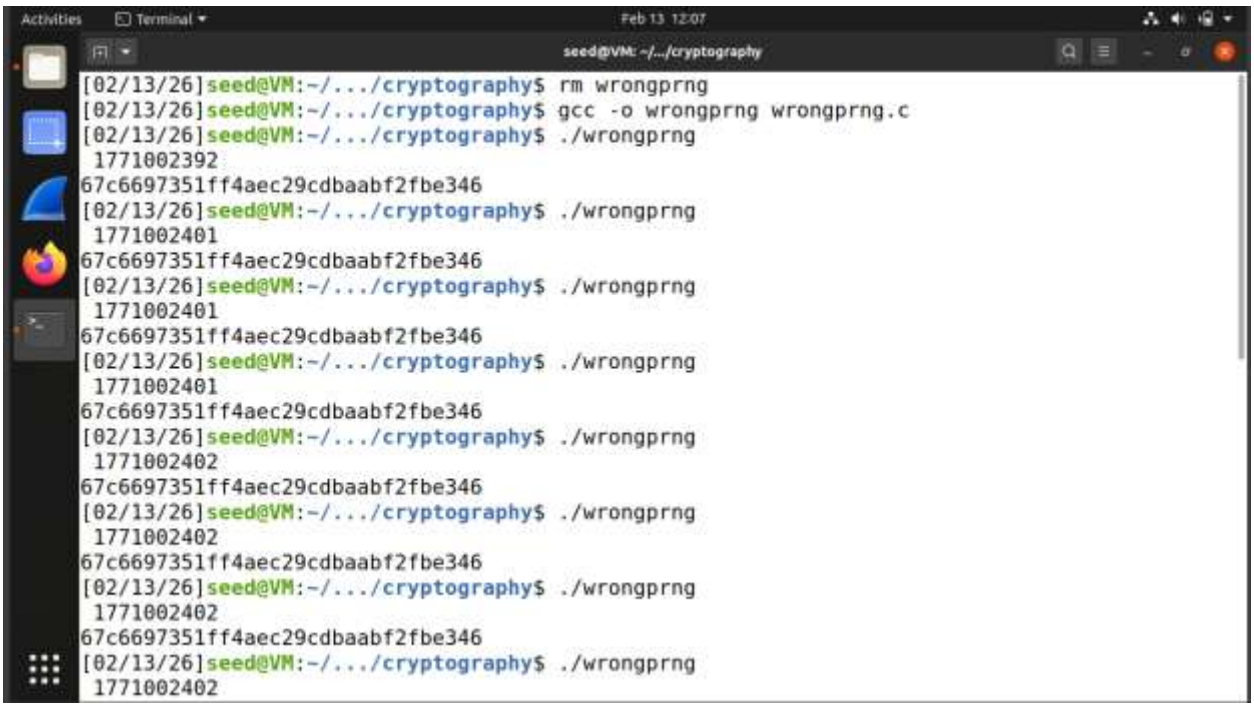


Figure 1.4: Output of PRNG without srand().

Running the program yields a 128-bit key as expected (Figure 1.1). Running the program in rapid succession withing the same seconds shows that the same key is produced within the same second (Figure1.2). This is expected as the PRNG is producing randomness based off the time for the seed. That means that all keys generated with the same seed or, in this case, the same time window, will be the same. Commenting out the srand() function does not break the program as it is still able to run, however the keys the program produces are all the same (Figure1.3,1.4). This is because the function commented out, srand() is where the seed for the PRNG was coming from. Without the seed, the PRNG will generate the same key, most likely defaulting to some set seed within the function call itself. This reveals the purpose of both

srand() and time() within srand(). The function srand() is what was feeding the PRNG seeds to produce different keys. The argument passed into the function was time(). This function outputs a timestamp and by feeding this timestamp into srand(), the PRNG can use time as a seed to produce different keys based off the time of generation.

Task 2

Objective

Create a program that a is able to brute force guess the correct AES-128-CBC key given known plaintext, cyphertext, and initial vector.

Instructions

1. Using the known plaintext, ciphertext, IV, and file timestamp, write a program that iterates through a two-hour timestamp window to regenerate possible keys and recover the correct AES-128-CBC key.
2. Include code snippets, recovered key, and explanation of why the attack works.

Observations

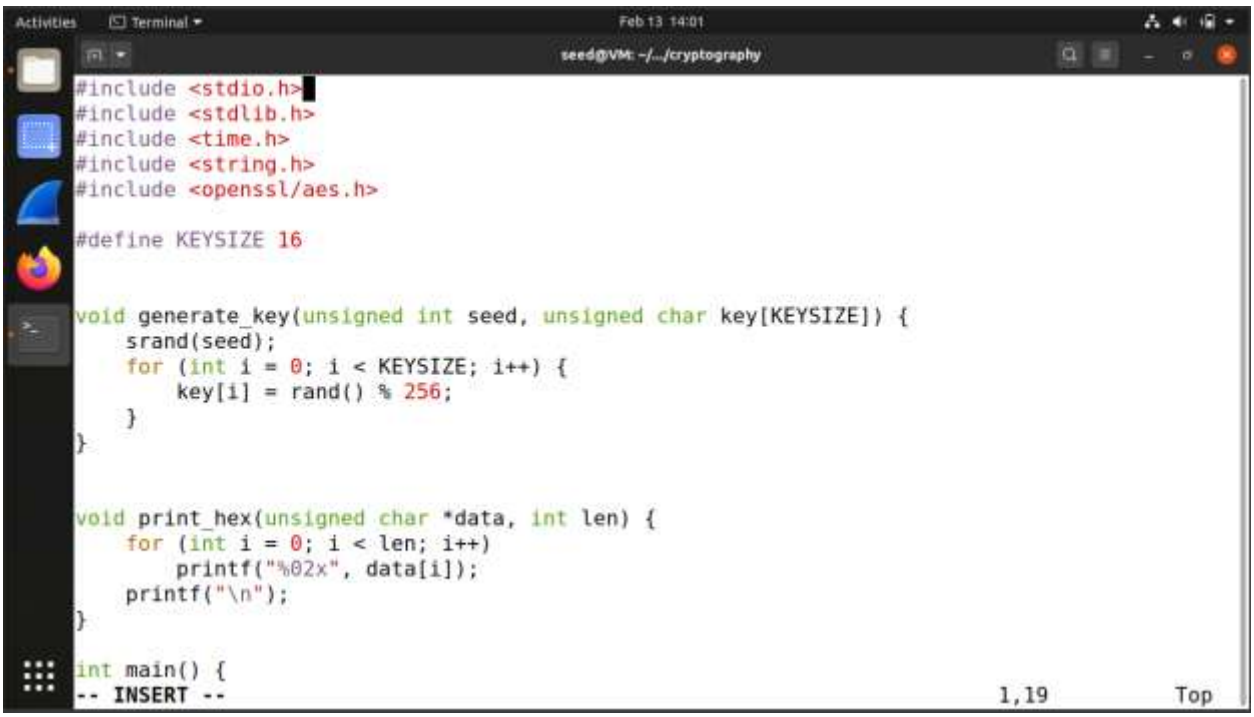


Figure 2.1: Program source code.

```
int main() {

    // Known values
    unsigned char plaintext[16] = {
        0x25,0x50,0x44,0x46,0x2d,0x31,0x2e,0x35,
        0x0a,0x25,0xd0,0xd4,0xc5,0xd8,0x0a,0x34
    };

    unsigned char ciphertext_target[16] = {
        0xd0,0x6b,0xf9,0xd0,0xda,0xb8,0xe8,0xef,
        0x88,0x06,0x60,0xd2,0xaf,0x65,0xaa,0x82
    };

    unsigned char iv[16] = {
        0x09,0x08,0x07,0x06,0x05,0x04,0x03,0x02,
        0x01,0x00,0xA2,0xB2,0xC2,0xD2,0xE2,0xF2
    };

    // File timestamp (epoch)
    time_t target_time = 1524006529; // 2018-04-17 23:08:49 approx

    unsigned char key[KEYSIZE];
    unsigned char encrypted[16];
                                                    41,0-1        38%
```

Figure 2.2: Program source code.

```
    unsigned char encrypted[16];
    AES_KEY aes_key;

    printf("Starting brute force...\n");


    for (time_t t = target_time - 43200; t <= target_time + 43200; t++) {

        generate_key((unsigned int)t, key);


        AES_set_encrypt_key(key, 128, &aes_key);


        unsigned char block[16];
        memcpy(block, plaintext, 16);



            block[i] ^= iv[i];

        AES_encrypt(block, encrypted, &aes_key);

        // Compare result
                                                    64,0-1        78%
```

Figure 2.3: Program source code.

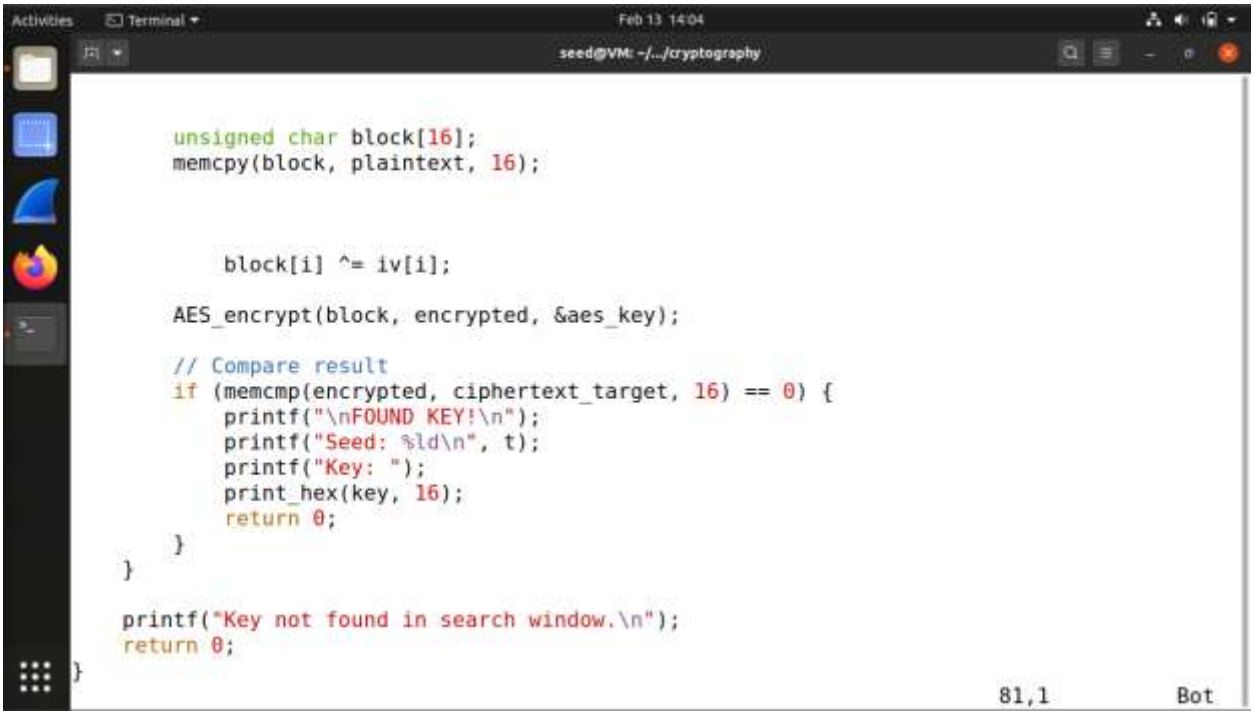


Figure 2.4: Program source code.

Figure 2.5: Program output.

The process of writing program that brute force guessed the keys involved first translating the time stamp into epoch, then it was a matter feeding the timestamp into the AES encryption algorithm and comparing the created key to the known key of the pdf file. This process runs in a for loop to iterate through timestamps that corresponds to a 2 hour time window (Figure 2.1-4).

Running the program shows how quickly the key is able to be guessed. As expected, the program is able to correctly find the key that matches the encryption of the pdf file (Figure 2.5).

Task 3

Objective

Observe how much entropy the kernel has at the current moment.

Instructions

1. Run watch -n .1 cat /proc/sys/kernel/random/entropy_avail into the terminal
2. Perform various activities (idle, mouse movement, typing, reading files) and observe entropy changes. Describe which activities increase entropy most and explain why.

Observation

Figure 3.1: Command to show entropy of the system.

Figure 3.2: Output of the command.

This command showed entropy changes as different hardware and software changed were detected (Figure 3.1). The more inputs that were given, the higher the number indicating higher entropy (Figure 3.2). The activities that changed and raised the entropy the most were mouse movements as the exact position of the cursor was never the same and it was very hard to predict the exact location of the courser at any given time.

Task 4

Objective

Explore the limitations of /dev/random

Instructions

1. Run: cat /dev/random | hexdump
2. Monitor entropy simultaneously.
3. Explain blocking behavior and describe how a Denial-of Service attack could be launched if a server relies on /dev/random for session key generation.

Observations

Figure 4.1: Running cat/dev/random | hexdump.

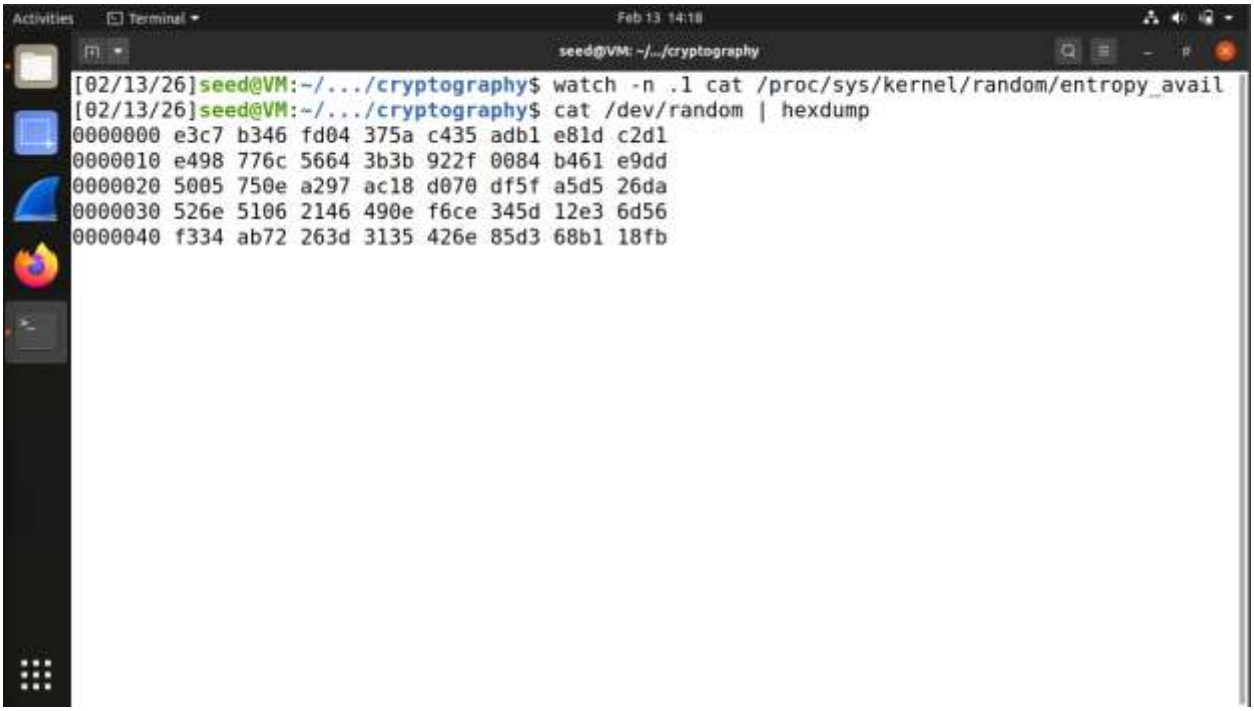


Figure 4.2: Output of the command.

The execution of cat /dev/random | hexdump reveals that /dev/random is a blocking device that stops output when the system entropy pool is depleted (Figure 4.1, 4.2). While the system remains idle, the output pauses until physical activity—such as mouse movement or keyboard input—refills the pool. Monitoring /proc/sys/kernel/random/entropy_avail confirms that entropy levels decrease during reads and increase during these physical interactions. Consequently, the generation of random bytes is strictly limited by the availability of environmental noise.

A server relying on /dev/random for session keys is vulnerable to Denial-of-Service attacks through entropy exhaustion. An attacker can trigger numerous connection requests to rapidly drain the entropy pool, causing the server to block and become unresponsive. This prevents the server from completing handshakes for legitimate users until sufficient randomness is gathered

Task 5

Objective

Evaluate the non-blocking behavior of /dev/urandom and analyze its statistical quality using the ent utility.

Instructions

1. Generate 1MB of data from /dev/urandom and analyze it using ent.
2. Modify the provided code to generate a 256-bit key.
3. Compile and execute your program.
4. Compare /dev/random and /dev/urandom and explain why /dev/urandom is recommended in practice.

Observations

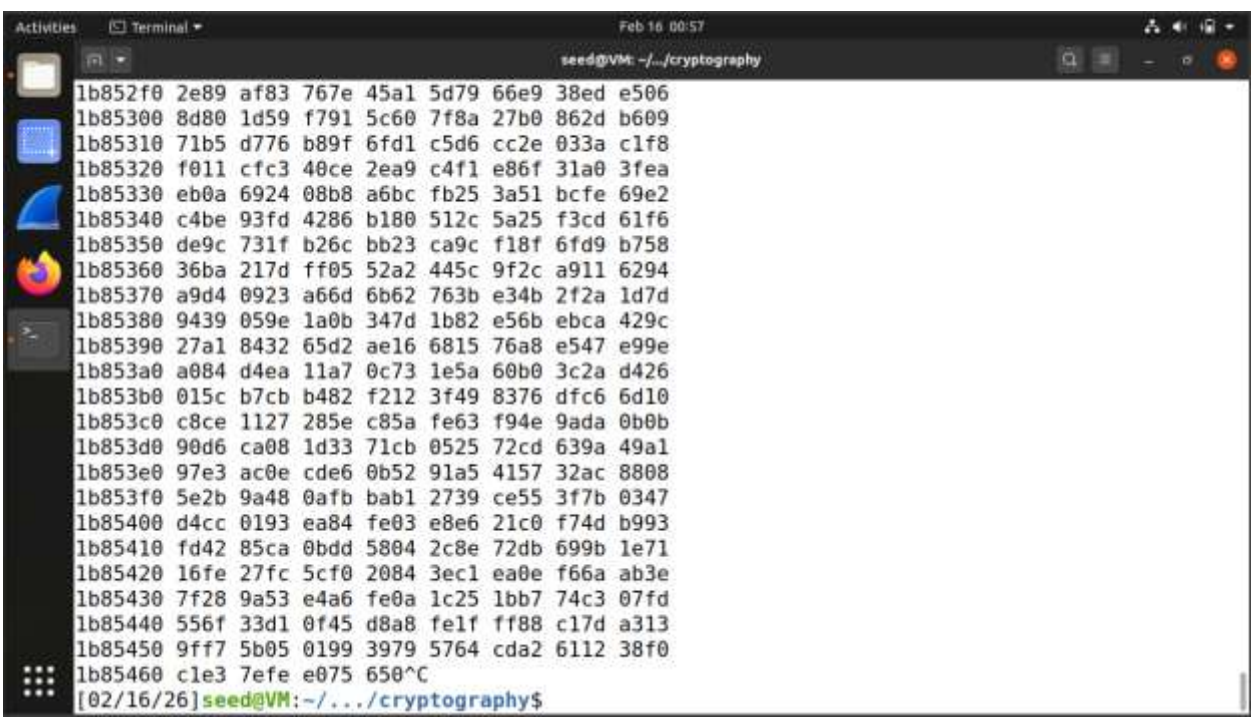
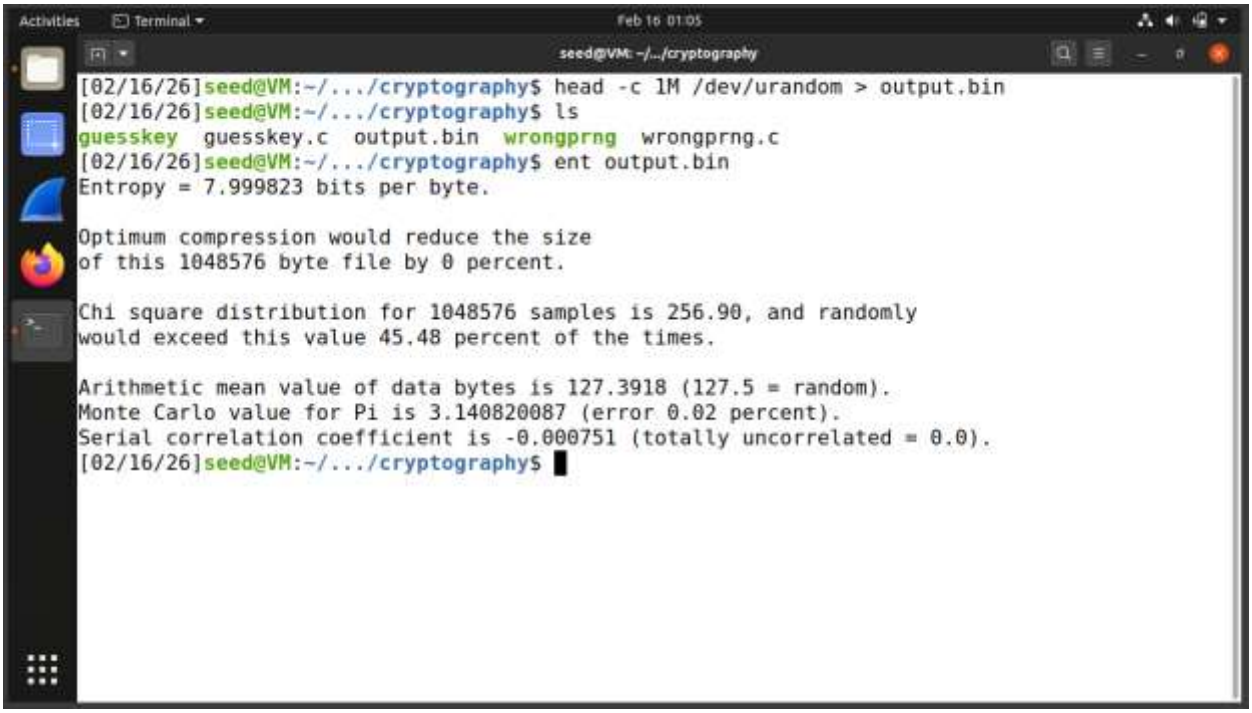


Figure 5.1: Output hexdump of /dev/urandom.

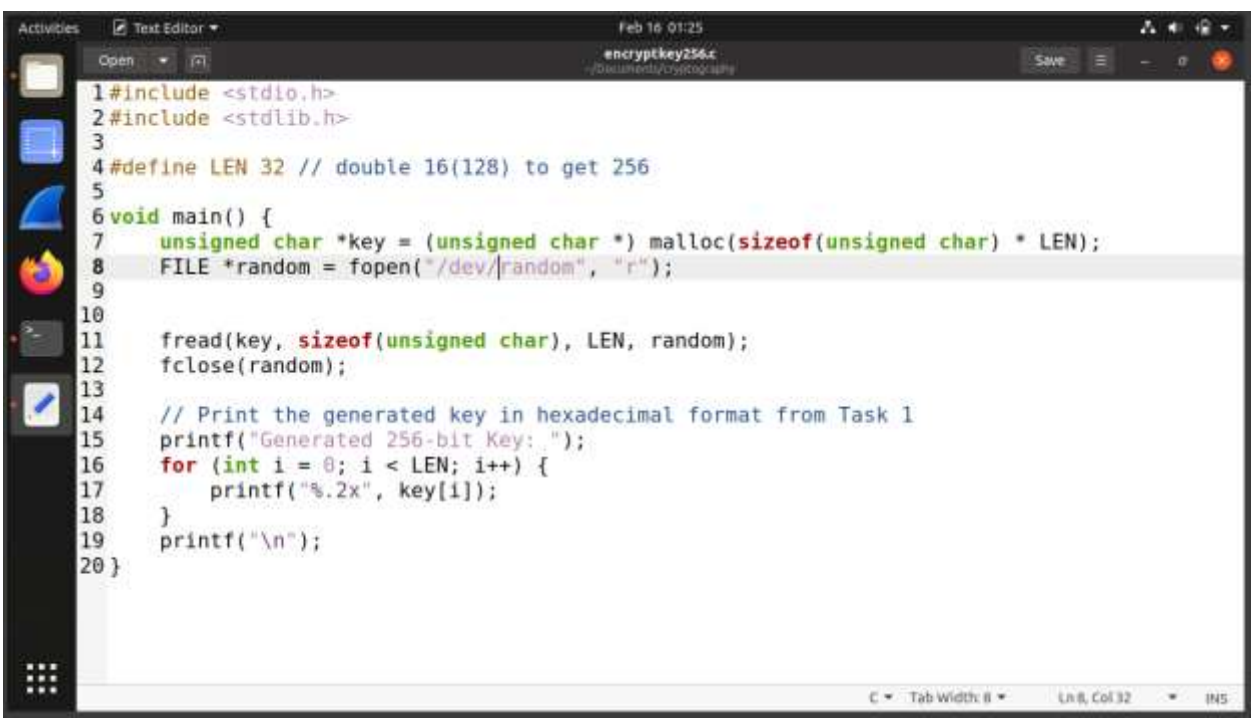
Running the command, “cat /dev/urandom | hexdump”, it is obvious the difference between this hexdump and the hexdump generated in Task 4 with /dev/random. In this hexdump, the output is continuous (Figure 5.1). There is no clear moment where inputs with the mouse or keyboard are obvious.



```
[02/16/26]seed@VM:~/.../cryptography$ head -c 1M /dev/urandom > output.bin
[02/16/26]seed@VM:~/.../cryptography$ ls
guesskey  guesskey.c  output.bin  wrongprng  wrongprng.c
[02/16/26]seed@VM:~/.../cryptography$ ent output.bin
Entropy = 7.999823 bits per byte.

Optimum compression would reduce the size
of this 1048576 byte file by 0 percent.

Chi square distribution for 1048576 samples is 256.90, and randomly
would exceed this value 45.48 percent of the times.

Arithmetic mean value of data bytes is 127.3918 (127.5 = random).
Monte Carlo value for Pi is 3.140820087 (error 0.02 percent).
Serial correlation coefficient is -0.000751 (totally uncorrelated = 0.0).
[02/16/26]seed@VM:~/.../cryptography$
```

Figure 5.2: Output of ent.

Creating a .bin file for /dev/urandom and running ent on the output file it can be surmised that the quality of the output numbers is high (Figure 5.2). The ent analysis shows an entropy of 7.999823 bits per byte and a 0% compression rate, indicating that the data is virtually indistinguishable from a truly random sequence. With an arithmetic mean of 127.39 and a serial correlation coefficient nearly at zero, the byte distribution is highly balanced and uncorrelated. These results confirm that /dev/urandom produces high-quality cryptographic randomness suitable for secure applications.



```
#include <stdio.h>
#include <stdlib.h>

#define LEN 32 // double 16(128) to get 256

void main() {
    unsigned char *key = (unsigned char *) malloc(sizeof(unsigned char) * LEN);
    FILE *random = fopen("/dev/random", "r");


    fread(key, sizeof(unsigned char), LEN, random);
    fclose(random);

    // Print the generated key in hexadecimal format from Task 1
    printf("Generated 256-bit Key: ");
    for (int i = 0; i < LEN; i++) {
        printf("%.2x", key[i]);
    }
    printf("\n");
}
```

Figure 5.3: Program to generate a 256-bit encryption key.

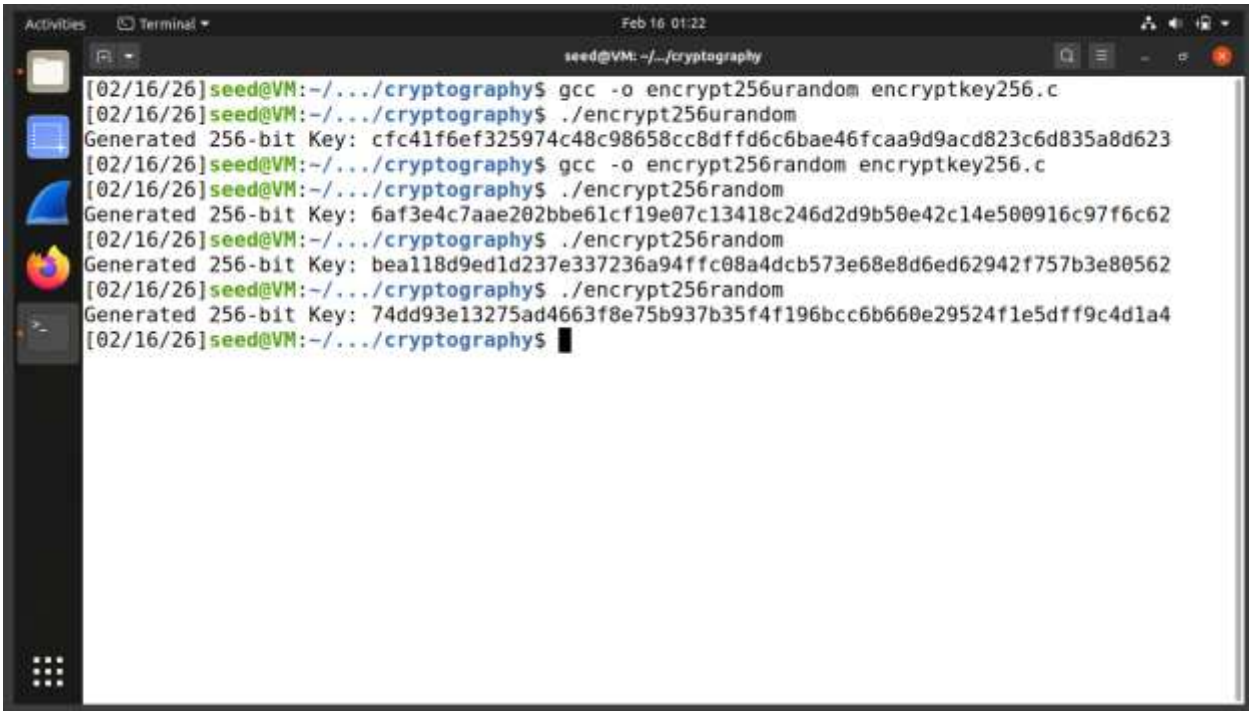


Figure 5.4: 256-bit encryption key output.

The program function as expected both with “/dev/urandom” and “/dev/random” (Figure 5.3). Both versions of the program generated keys that are equally complex and random (Figure 5.4). This means that in theory, both keys are able to provide equally high-quality keys suitable for AES encryption. Practically, however, while “/dev/urandom” provided the keys instantly, “/dev/random” required some time to generate sufficient entropy or have sufficient entropy beforehand before generating the key as seen in the past tasks. Even though both devices produce statistically strong keys, “/dev/urandom” is preferred because it guarantees a non-blocking response, ensuring the application does not stall or become vulnerable to Denial-of-Service attacks during key generation.